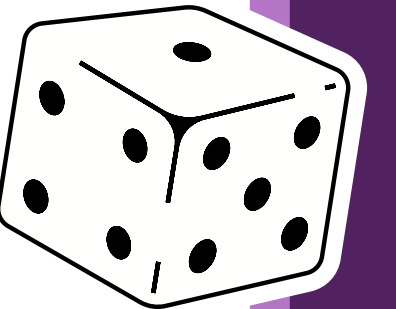
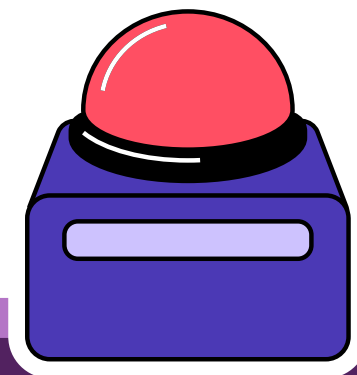


BCC1244 - GAME PROGRAMMING



# GAME LOOP AND GAME MECHANICS



START



# INTRODUCTION

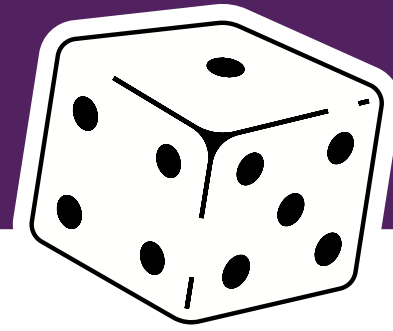
## Introduction to Game Development

- *Games are interactive systems that require real-time processing of player input, game logic, and rendering.*

## Key components of game development:

- *Game Loop: The heartbeat of the game.*
- *Game States: The different modes or phases of the game.*
- *Core Mechanics: The rules and systems that define gameplay.*





# THE GAME LOOP

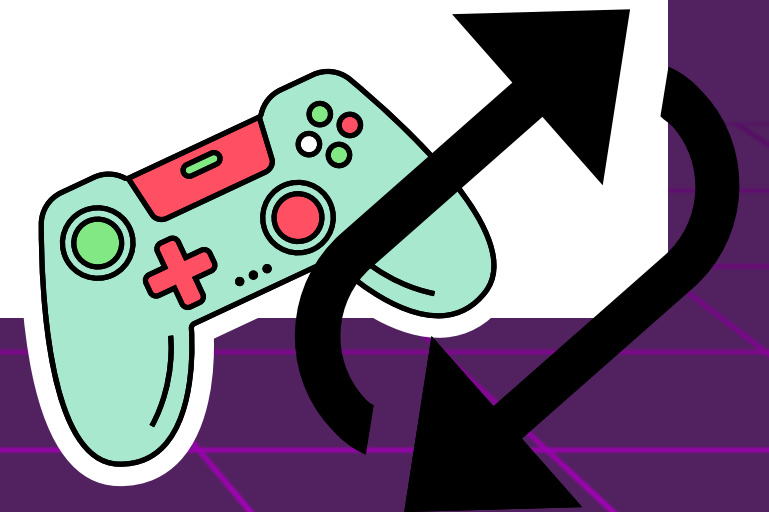
# THE GAME LOOP

## Definition:

- *The game loop is a continuous cycle that updates the game state and renders the game frame-by-frame.*

## Purpose:

- *Ensures the game runs smoothly and responds to player input in real-time*



# THE GAME LOOP

## Structure of a Game Loop:

python

Copy

```
while game_is_running:
    process_input() # Handle player input
    update_game_state() # Update game logic
    render() # Draw the current frame
```

## Key Concepts:

- *Frame Rate (FPS): The number of frames rendered per second. Higher FPS means smoother gameplay.*
- *Delta Time: The time elapsed since the last frame. Used to ensure consistent movement regardless of frame rate.*
- *Fixed Time Step: A technique to update game logic at a consistent rate, independent of rendering.*

# THE GAME LOOP

Frame Rate Per Second (FPS)



GeForce Powered High FPS CS:GO SLO-MO Video

240 FPS Hz

144 FPS Hz

60 FPS Hz

NVIDIA

Watch on YouTube

[CLICK HERE](#)

# THE GAME LOOP

Delta Time

`Time.deltaTime = 0`

`Time.deltaTime = 0.05;`

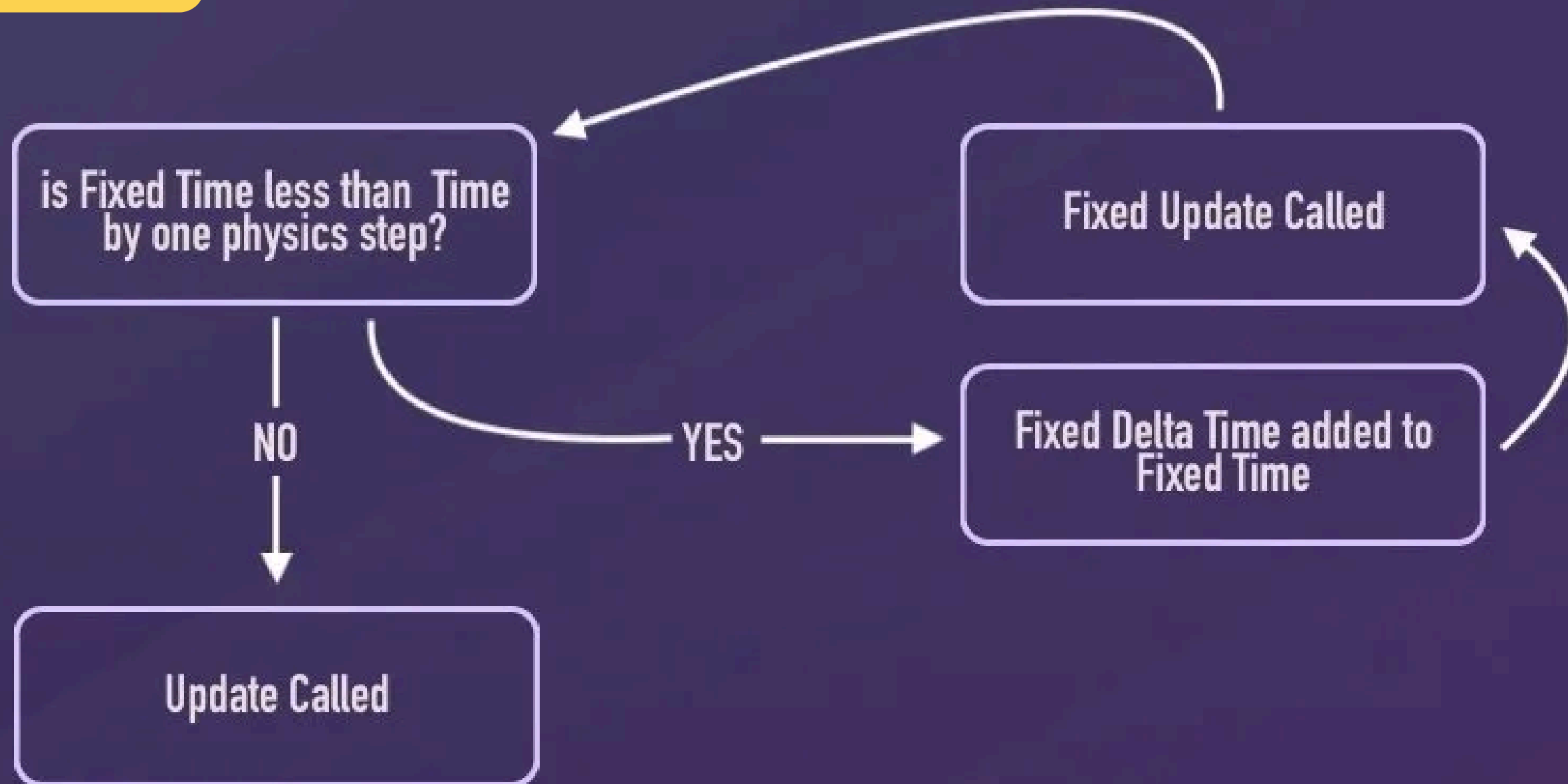
frame 1

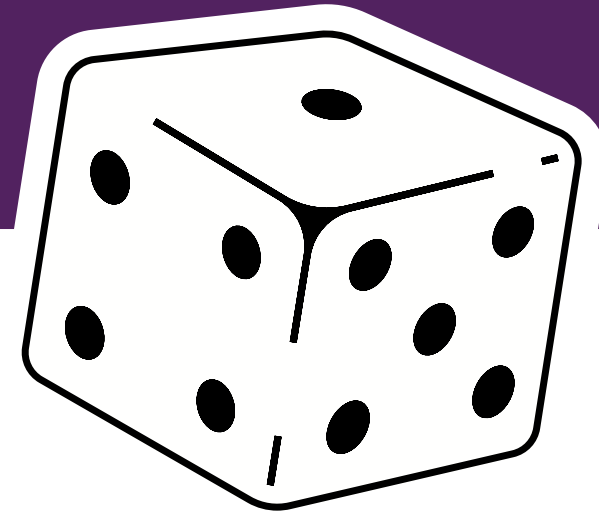
frame 2

Time passed between these 2  
frames is 0.05

# THE GAME LOOP

## Fixed Time State





GAME STATES

START

# GAME STATES

## Definition:

- *Game states represent the different modes or phases of a game (e.g., Main Menu, Playing, Paused, Game Over).*

## Importance:

- *Helps manage the flow of the game and organize logic for different scenarios.*



# GAME STATES

## Common Game States:

- *Main Menu: Where players start the game or adjust settings.*
- *Playing: The core gameplay state.*
- *Paused: Temporarily halts gameplay.*
- *Game Over: Indicates the end of the game (e.g., player loses).*
- *Cutscenes: Non-interactive story segments.*

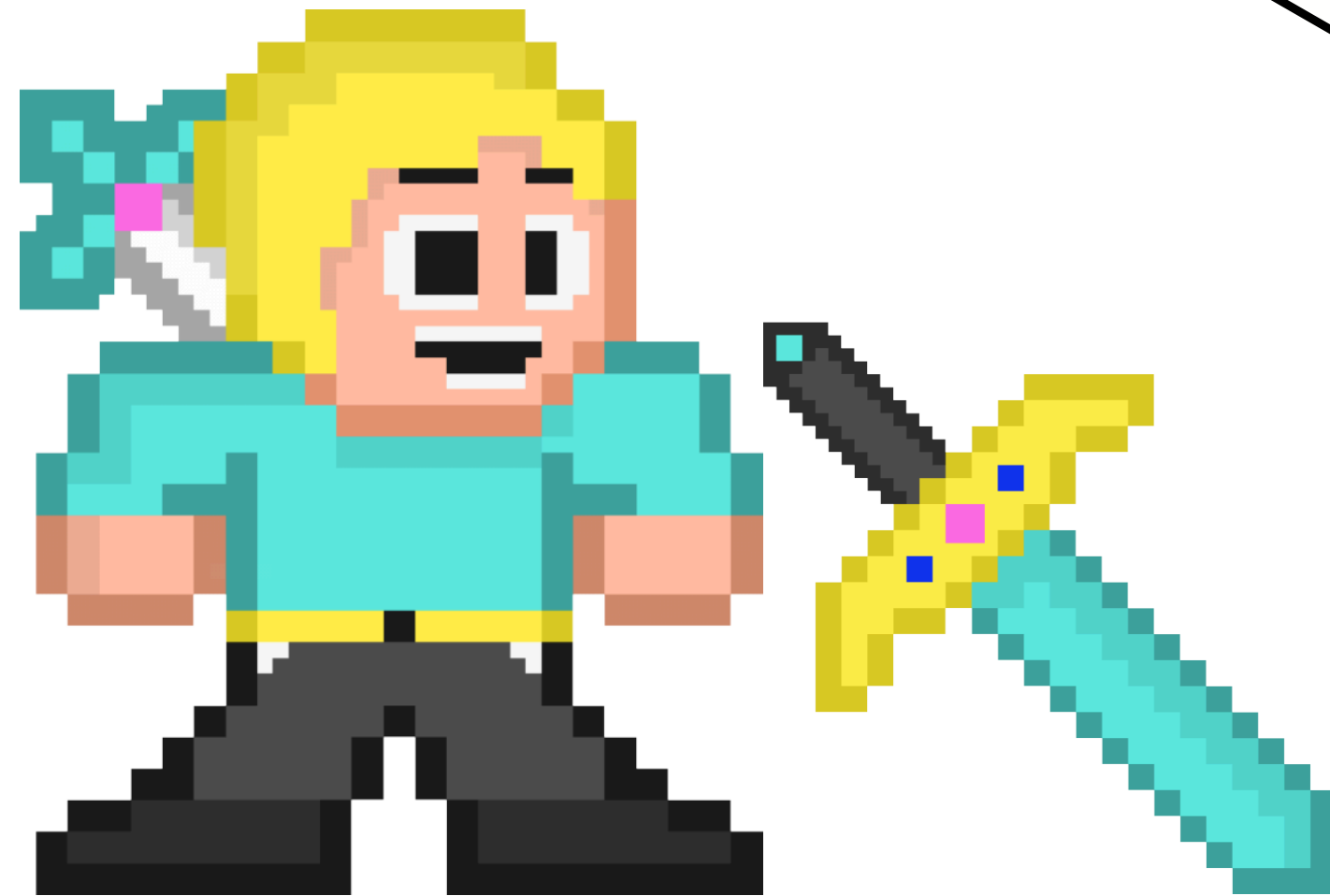
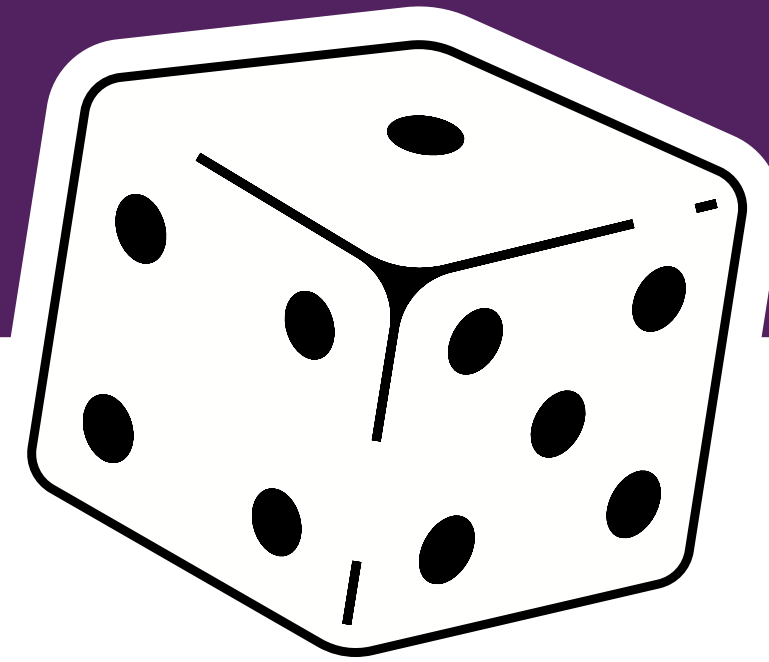
## State Management:

- *Use a state machine to transition between states.*
- *Example:*

python

Copy

```
if player_presses_start:
    current_state = "Playing"
elif player_pauses:
    current_state = "Paused"
```



# CORE MECHANICS

# CORE MECHANICS

## Definition:

- *Core mechanics are the fundamental rules and systems that define how the game works.*

## Examples:

- *Movement: How the player character moves (e.g., walking, jumping).*
- *Combat: How players fight enemies (e.g., attacking, blocking).*
- *Progression: How players advance through the game (e.g., leveling up, unlocking new areas).*

# CORE MECHANICS

## Design Principles:

- *Consistency: Mechanics should follow clear, predictable rules.*
- *Feedback: Players should receive immediate feedback for their actions (e.g., sound effects, visual cues).*
- *Balance: Mechanics should be fair and challenging but not frustrating.*

## Iterative Design:

- *Prototype mechanics early.*
- *Test and refine based on player feedback.*

# CORE MECHANICS

[Click Here](#)



What are video game mechanics? Concept explained!



Share

## WHAT ARE VIDEO GAME MECHANICS?

Watch on  YouTube

# INTEGRATING THE GAME LOOP, GAME STATES, AND CORE MECHANICS

- *The game loop drives the execution of game states and core mechanics.*
- *Example Integration:*

python

Copy

```
while game_is_running:
    process_input()

    if current_state == "Playing":
        update_game_state() # Update core mechanics (e.g., movement, combat)
    elif current_state == "Paused":
        display_pause_menu()

    render()
```



THANK YOU  
FOR PLAYING!

